Authored by: Arpit Raj, LNMIIT Jaipur
Date: October 22, 2026
© 2026 Arpit Raj. All rights reserved.

Object-Oriented Programming: Core Concepts & Constructors

Introduction to OOPs

OOPs is a programming style that divides a program into pieces of object that can communicate with each other.

- Object based coding style -> every object have its own attributes and behaviour.
- A modular approach :- OOPS lets us organize program into classes which encapsulates related data and ops to be performed on those data.
- -> this allow different part of programs to be developed and maintained independently also enhances code reusability.

Diagram: Class to Objects

Class: Student

Contains attributes `name`, `roll`, `CG` and methods `func1()`, `func2()`, `func3()`.

It branches out to 5 instances:

Instance	Name	Roll	CG
s1	Gham	023	5.4
s2	A	015	7.4
s3	B	067	7.1
s4	Arpit	011	7.9
s5	Aadya	006	8.0

> **Note:**
> s1, s2, s3, s4, s5 are objects.

Classes vs Objects

- Object is an instance of Class
- Class is blueprint of object
- **Attributes** :- state, properties of an object
- **Behaviour** :- methods, functions that object can perform

Basic construction of class

```
class Name {public:
    attributes
    behaviours
    constructor
    destructor
};
```

Eg. Now as soon as you try to create an instance of this class by

```
Name A;
```

constructor is called, and when return is executed in main, destructor is called by default.

- -> name of constructor should be same as class name
- -> name of destructor should be same as `~class name`
- `'~'` is important

Constructors & Destructors

Default constructor vs parametrized constructor

```
class Aadya {
    string name;
    int age;
    Aadya() {
        // constructor
    } // this is default constructor

    ~Aadya() {
        // destructor
    }
};

main() {
    Aadya A;
    A.name = "aadya";
    A.age = 21;
}
```

In a parametrized constructor

```
Aadya(string name, int age) {
    this->name = name;
    this->age = age;
}
```

then inside main we can write

```
Aadya("aadya", 21);
```

now the constructor will utilize these values and set the attributes of that class.

> **A destructor cant be parametrized**

Copy Constructors

-> one object of a class is made now I need to copy data of that object into another object.

Syntax :

```
Student(string name, int age) {
    this->name = name;
    this->age = age;
    // Normal parameter constructor
}

Student(const Student &srcobj) {
    this->name = srcobj.name;
    this->age = srcobj.age;
}
```

now in main

```
Student A("aadya", 21);
Student B = A;
```

basically we are passing object of A of student class as a parameter
`Student B = A` or `Student B(A)`

Static vs Dynamic Allocation

I. (Static) / stack allocation

```
Student a("aadya", 21);
```

- `a` is an object with automatic storage duration, lifetime ends when execution leaves scope
- `cout << a.name`

II. (Dynamic) / heap allocation

```
Student *a = new Student("aadya", 21);
```

- dynamic allocation requests memory at runtime using `'new'` in cpp, object remains until deleted like `(delete a)`
- `cout << a->name` (because `a` is pointer)
- in dynamic allocation, destructor isnt called until `delete` is called

Note / quick question

```
int age;
int *gpa;

Student {
    this->age = age;
    this->gpa = new int(gpa); // (:: it's a pointer)
}

~Student {
    delete this->gpa; // since dynamically allocated, needs to be deleted
too
}
```